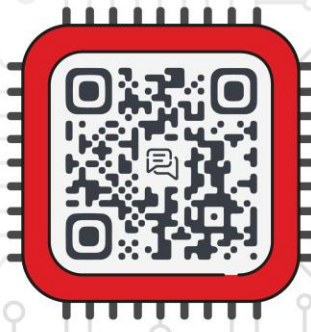




نظري

9



13



1720 sp

كلية الهندسة المعلوماتية

السنة الثالثة

Fully Associative Cache

د. سيرا أستور



RB Informatics; 15/12/2025

بنيان الحواسيب 2

أفكار المحاضرة:

- Fully Associative Cache
- Set-Associative Cache
- Two-Way Set-Associative Cache
- Set Associative Cache – Summary
- Replacement Strategies
- Handling Writes
- Designing Memory System
- Interleaved Memory System

تحدثنا في المحاضرة السابقة عن الذاكرة المؤقتة (Cache) التي تُعتبر جزءاً من Hardware حيث تصل Hit Rate إلى حوالي 95% مما يجعلها سريعة جداً وفي مسانئنا سنتعامل مع Miss Rate, Hit Rate

كما ذكرنا أن الذاكرة الرئيسية (Main Memory) يتعامل معها فقط نظام التشغيل (Operating System) وتختلف الذاكرة المؤقتة عن الذاكرة العشوائية (RAM) بأنها تخزن عناوين (Tags) قد تكون العنوان كامل أو جزءاً منها وأشارنا إلى أن الذاكرة العشوائية عند الإقلاع تحتوي على قيم عشوائية وهذا ينطبق أيضاً على الذاكرة المؤقتة مما يؤدي إلى ارتفاع Miss Rate عند بدء تشغيل الجهاز.

تذكير: نحن لا نتعامل مع ال Performance بحد ذاته وقد ذكرنا ذلك منذ مادة البنيان 1 وانما نتعامل مع Performance Cost أي يهمننا ان يكون الأداء مقبول والسعر مناسب لذلك نسعى دائما الى التحسين بمقدار مناسب



ومن طرق تحسين أداء الكاش:

1. زيادة حجم Cache:

عندما نتحدث عن زيادة حجم الذاكرة المؤقتة فإننا نعني توسيع سعة الذاكرة هذا يعني أنه يمكن تخزين المزيد من البيانات في وقت واحد

2. زيادة حجم الـ Block:

حجم الكتلة هو مقدار البيانات التي يتم تحميلها من الذاكرة الرئيسية إلى الذاكرة المؤقتة في عملية واحدة فعند زيادة حجم الكتلة يتم تحميل المزيد من البيانات في كل مرة

Cash Types

Direct Cashe

تحدثنا عن الطريقة الأولى وهي Direct Cache حيث يتم تقسيم العنوان إلى ثلاثة أجزاء:

1. Index

2. Tag

3. Offset

لكننا وجدنا أنه في حال تكرار رقم السطر سيتم استبدال محتواه مما يعني أننا نكتب على نفس السطر عدة مرات دون الاستفادة من حجم الذاكرة المؤقتة

💡 لذا وجدنا طريقة أفضل من حيث التعامل ولكنها ستكون أكثر تكلفة والتي سنتناولها في هذه المحاضرة وهي الذاكرة المؤقتة الشاملة (Fully Associative Cache)

Fully Associative Cache

هي طريقة تخزين وتنظيم البيانات في الكاش بحيث أي Block من الذاكرة يمكن وضعه في أي Line في الكاش والفرق بينها وبين الأنواع الأخرى (Direct Mapped أو Set Associative) هو مرونتها في تحديد مكان وضع الـ Block حيث لا يوجد Index ويتم تقسيم العنوان إلى جزأين:

1. Tag

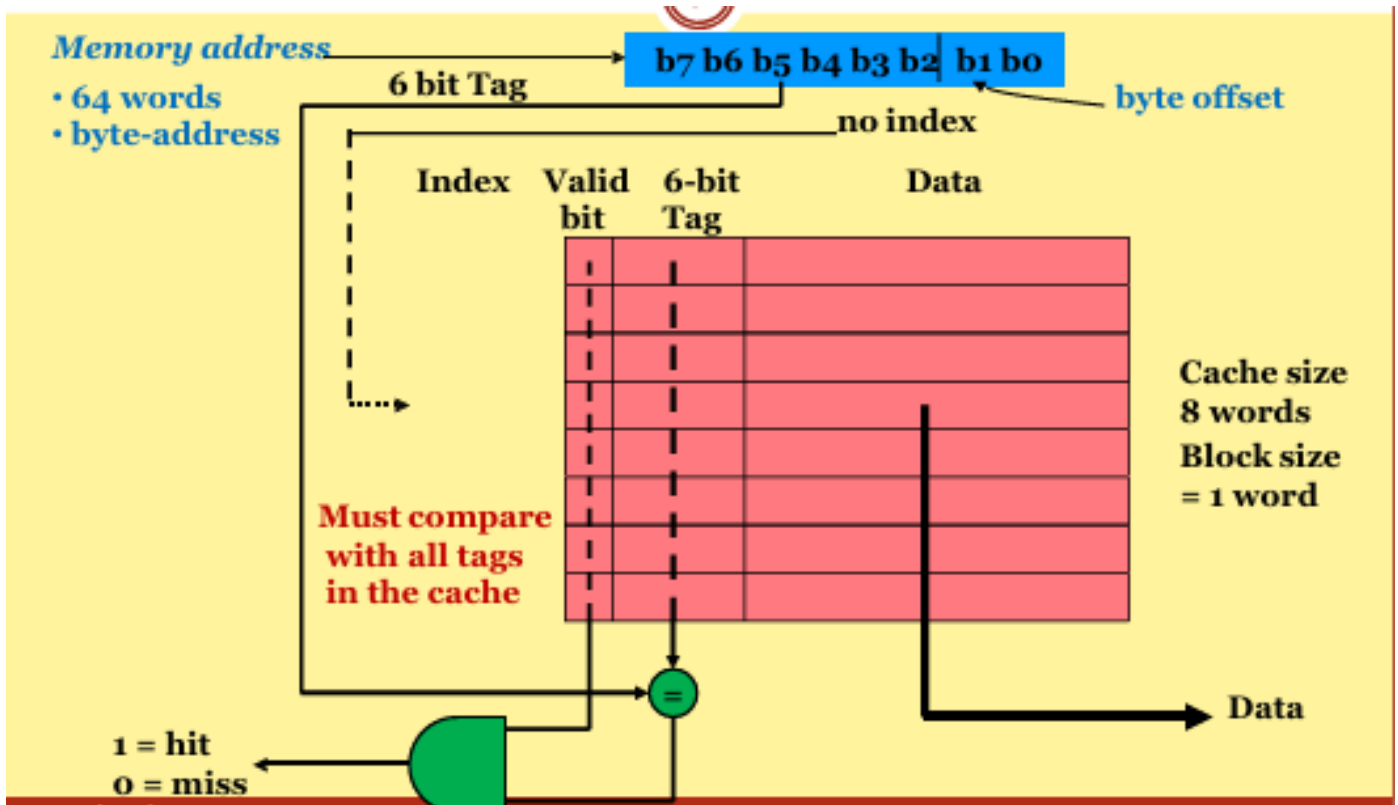
2. Offset

كيفية البحث عن البيانات:

- يتم استخراج الـ Tag والـ Offset من العنوان المطلوب ويتم مقارنة الـ Tag مع جميع الـ Tags الموجودة في الكاش في نفس الوقت وتكون العملية تتم بشكل متوازي (Parallel) إذا وجد تطابق يكون ذلك Cache Hit ومن ثم يتم الوصول إلى البيانات باستخدام الـ Offset داخل الـ Block عند حدوث Cache Hit
- ولكن في حالة Cache Miss يتم جلب الـ Block من الذاكرة الرئيسية إلى أي Line متاحة وإذا كان الكاش ممتلئاً يتم استبدال أحد الـ Lines حسب سياسة الاستبدال مثل LRU أو Random Replacement التي سيتم شرحها لاحقاً بالتفصيل

Direct-Mapped vs Fully Associative

Direct-Mapped	Fully Associative
One Comparator	N Comparators
رخيص	غالي
يوجد Conflict Miss (لأن كل Block من الذاكرة يذهب إلى Line محدد حسب الـ Index وإذا حاول Block جديد وضع نفسه في Line مشغول بالفعل بـ Block آخر، حتى لو كان الكاش فيه مساحة فارغة في Lines أخرى يحدث Conflict Miss)	لا يوجد Conflict Miss (في Fully Associative Cache لا يوجد Index محدد أي أنه Block يمكن أن يذهب لأي Line فارغ لذلك احتمالية Conflict Misses شبه معدومة)
لا يحتاج Replacement	يحتاج Replacement



Set-Associative Cache

- الـ Set-Associative Cache هي طريقة لتصميم الكاش تحدد أين يوضع كل Block من الذاكرة في الكاش، وتجمع بين بساطة Direct Mapped Cache ومرونة Fully Associative Cache (تعتبر الحل الوسط).
- تعتمد أن الكاش كله مُقسم إلى عدة Sets متساوية الحجم (مجموعة من Lines في الكاش) وكل Block من الذاكرة يمكن وضعه في Set واحدة فقط، يتم تحديدها يتم بواسطة Index ولكن يمكن أن يوضع في أي Line داخل هذه الـ Set.

تقسيم عنوان الذاكرة في Set-Associative Cache:

$$\text{Memory Address} = \text{TAG} | \text{INDEX} | \text{OFFSET}$$

الـ Index: يحدد الـ Set التي يذهب إليها الـ Block

الـ Tag: يُقارن مع كل Lines داخل الـ Set

الـ Offset: يحدد موقع البيانات داخل الـ Block

مثال:

- فرضا أن الكاش يحتوي على 8 Lines
- وكل Set فيها 2 Lines أي لدينا 4 Sets
- أي Block من الذاكرة يذهب إلى واحدة من هذه الـ 4 Sets الأربعة حسب الـ Index
- وبداخل الـ Set يمكن وضعه في أي Line متاحة من الاثنين

Two-Way Set-Associative Cache

- حيث نستخدم Set Index لتحديد أي Set
- نقارن الـ Tag مع 2-Tags لأنها 2-Way في هذا النوع من الكاش حيث تكون مقسمة إلى 2 Sets
- وكل Set تحتوي على 2 Lines لذلك تسمى 2-Way فعند وصول طلب من المعالج، نستخدم Set Index لتحديد أي Set يجب البحث فيها.
- بعد تحديد الـ Set نقارن الـ Tag المطلوب مع الـ 2 Tags للـ 2 Lines داخل الـ Set
 - إذا وجد تطابق Hit
 - إذا لا Miss
- ونحتاج Replacement يتم تقسيم العنوان:
 - Tag
 - Set Index
 - Offset

ونحتاج الى 2 Comparators لكل Set وMUX لاختيار البيانات الصحيحة تقسيم عنوان الذاكرة:

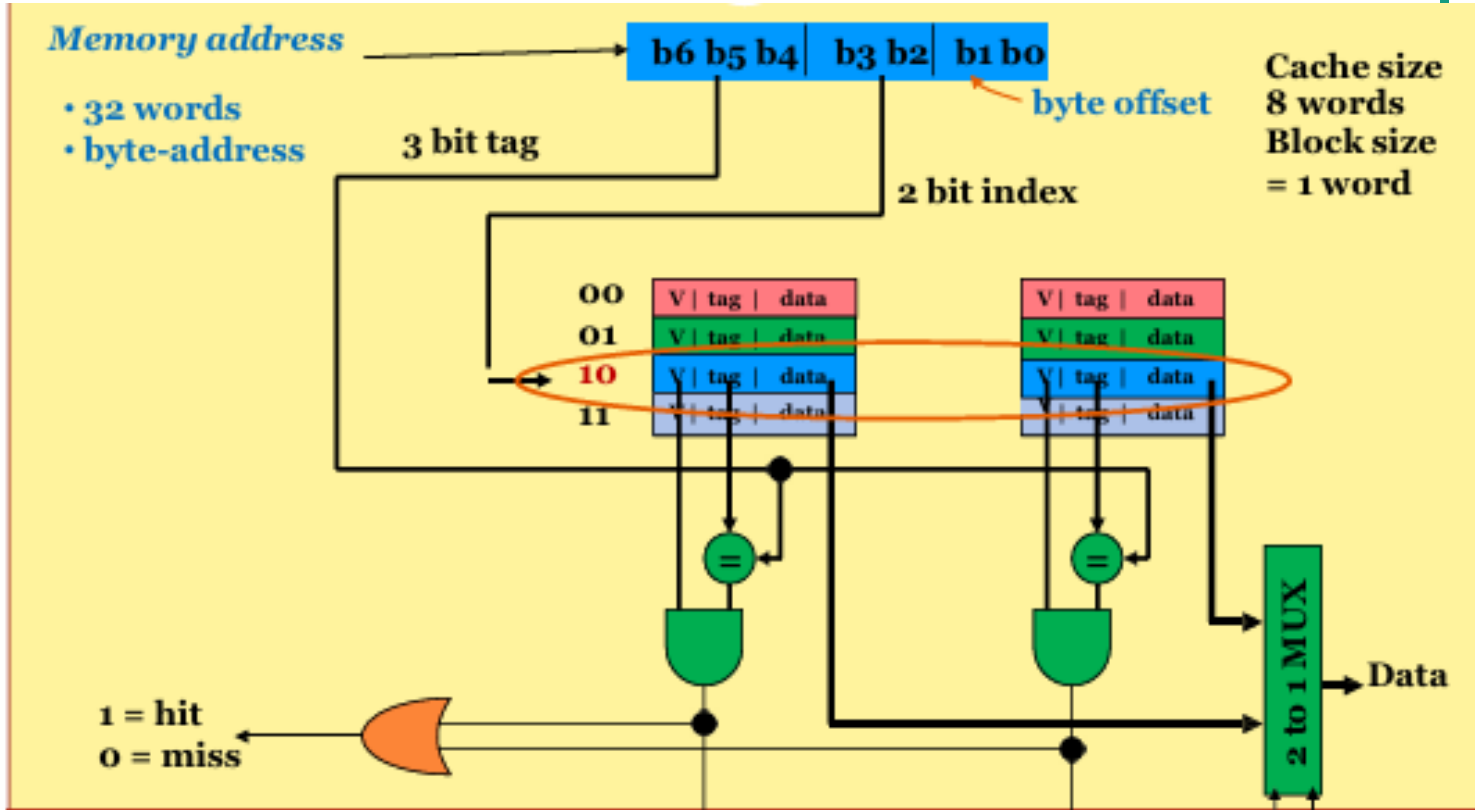
$$\text{Memory Address} = \text{TAG} | \text{SET INDEX} | \text{OFFSET}$$

Set Index لتحديد الـ Set التي ينتمي إليها الـ Block

وبما أن كل Set فيها 2 Lines نحتاج إلى 2 Comparators

إذا وجد تطابق Cache Hit وإذا لم يوجد تطابق Cache Miss

ونستخدم MUX (Multiplexer) لاختيار البيانات الموجودة في Line التي تم التطابق معها داخل الـ Set كما موضح بالرسم



Associativity vs Miss Rate

في الكاش يوجد علاقة بين درجة Associativity و Miss Rate:

1. زيادة Associativity تقلل الـ Conflict Misses

كلما زاد عدد Lines في الـ Set أو زادت Associativity أصبح لدى كل Block مزيد من الخيارات لوضع نفسه داخل الـ Set وهذا يقلل فرصة حدوث Conflict Misses لأنها تحدث عندما يريد Block معين مكان محدد مسبقاً في الكاش ويكون مشغولاً

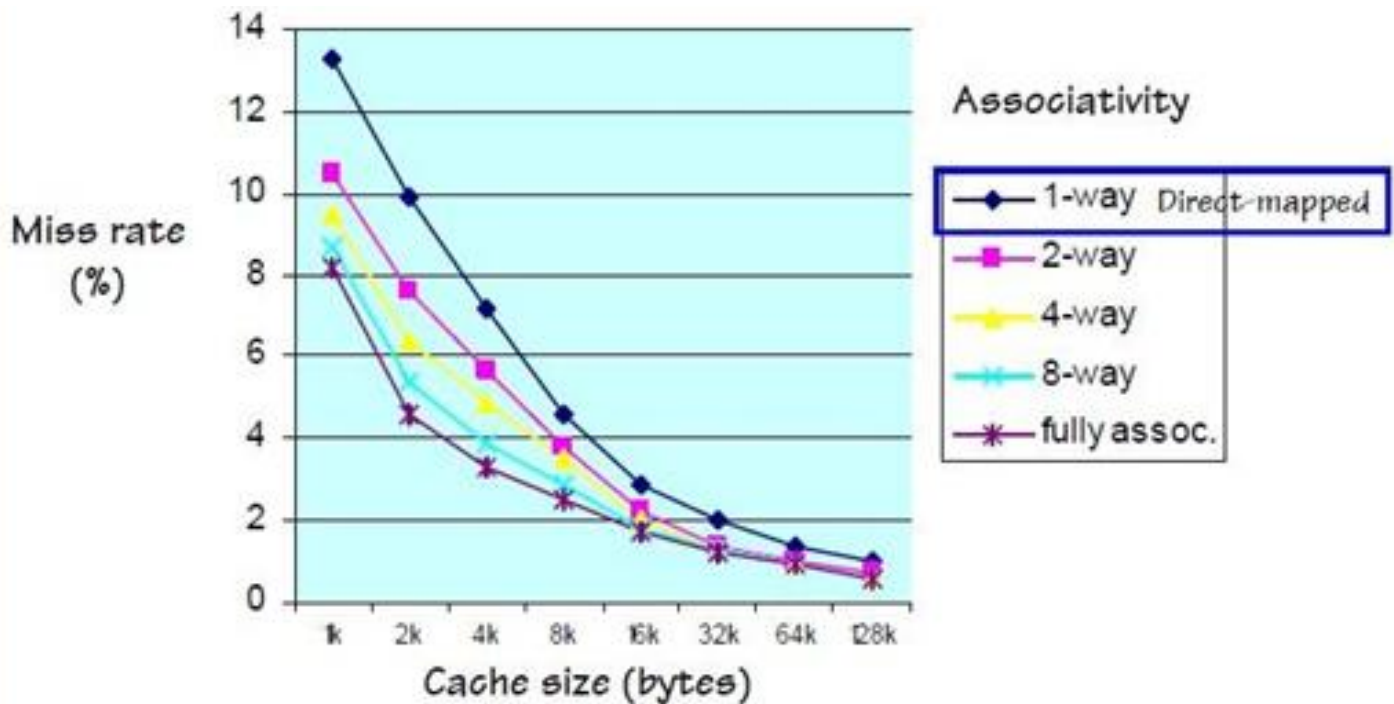
2. زيادة Associativity تزيد التعقيد

فمع زيادة عدد Lines في الـ Set يحتاج الكاش إلى مقارنة TAGs لكل Line داخل الـ Set وزيادة عدد Comparators يحتاج إلى عدد أكبر من MUX لاختيار البيانات الصحيحة عند Hit هذا يزيد حجم الدوائر والـ Hardware Complexity

3. زيادة Associativity قد تزيد وقت الوصول للكاش (Access Time)

مقارنة جميع TAGs داخل الـ Set واختيار البيانات المناسبة باستخدام MUX يحتاج وقتاً أطول لذلك هناك توازن مطلوب بين Associativity وتقليل الـ Miss Rate وبين سرعة الوصول وسهولة التصميم وخلاصة ذلك:

- Higher Associative -> Lower Conflict Misses -> Better Cache Hit Rate
- ولكن!
- Higher Associative -> More Access Time -> Complex Hardware
- لذلك عادة يُستخدم 2-Way Associative أو 4-Way Associative كحل وسط بين الأداء والتعقيد

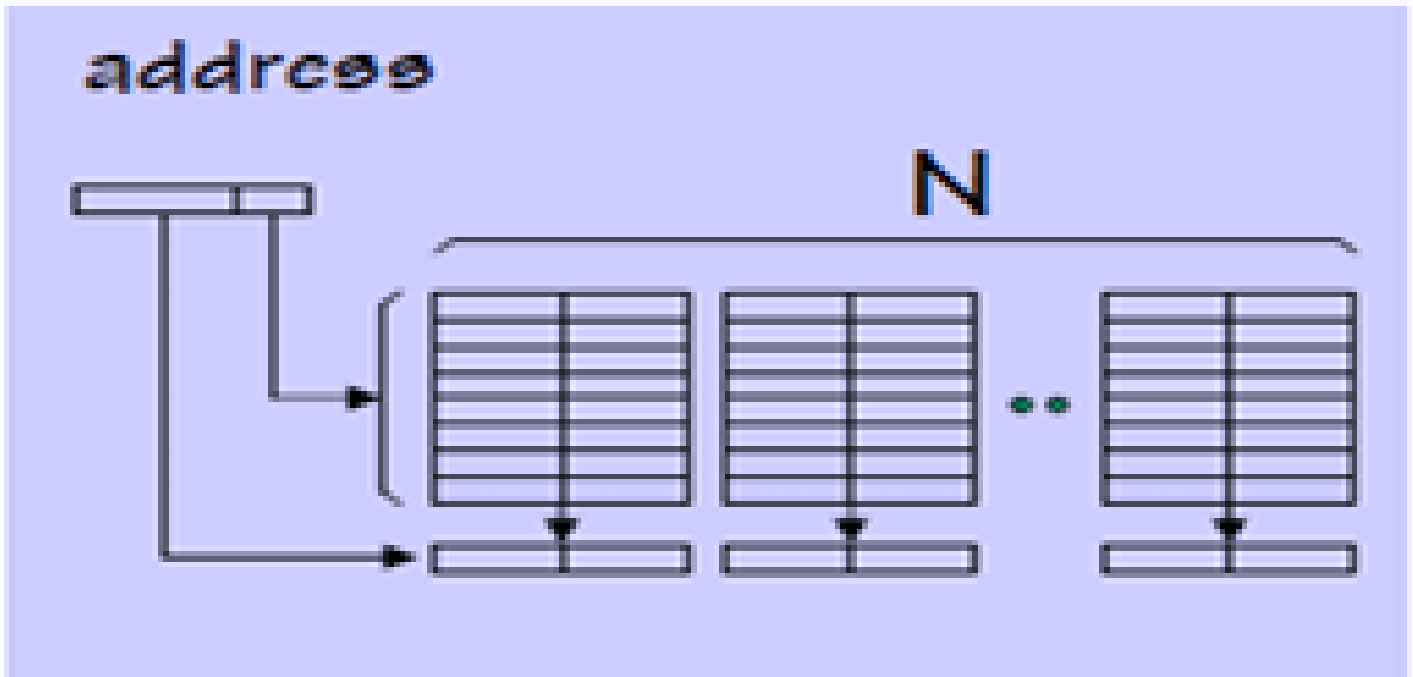


:Set Associative Cache – Summary

- يتم تنظيم الكاش بطريقة تقسيمها إلى Set وكل منها تحتوي على N Lines لذلك يسمى N-Way
- أي Block من الذاكرة الرئيسية يذهب إلى Set واحدة فقط (يتم تحديدها باستخدام Set Index)
- داخل هذه ال Set يمكن تخزين ال Block في أي Line من ال N Lines المتاحة
- يتم مقارنة العنوان مع N Tags والبيانات يمكن تخزينها في أي Line داخل Set وهذا يقلل من Conflict Misses مقارنة بال Direct-Mapped Cache، لأن ال Block ليس مقيداً بمكان واحد فقط

ويمكن النظر إلى N-Way Set Associative Cache كأنه:

- N Direct-Mapped Caches تعمل بالتوازي على نفس ال Set Index
- كل Way يعمل ك Direct-Mapped Cache مستقل وعند الوصول يتم فحص جميع هذه ال Ways في نفس الوقت ويتم اختيار النتيجة الصحيحة



Replacement Strategies (سياسات الاستبدال)

عند حدوث Cache Miss وكان الكاش ممتلئاً لا بد من حذف أحد الـ Blocks الموجودة لإفساح المجال للـ Block الجديد.

- Replacement Strategies تحدد أي Block سيتم حذفه.

1. LRU – Least Recently Used:

- يتم حذف الـ Block الذي لم يُستخدم منذ أطول فترة زمنية وتعتمد على مبدأ أن البيانات المستخدمة حديثاً غالباً ستُستخدم مرة أخرى قريباً تعطي أفضل Hit Rate في أغلب الحالات ولكن مكلفة من ناحية العتاد (Hardware) وتحتاج لتتبع استخدام كل Block (Counters أو Time Stamps).

2. FIFO / LRR – First in First Out:

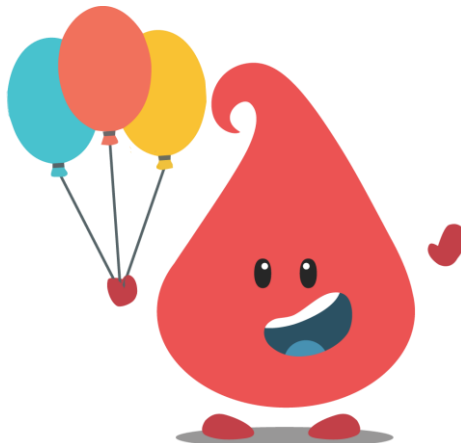
- يتم حذف أقدم Block تم إدخاله إلى الكاش بغض النظر عن استخدامه.
- أبسط من LRU وأسهل في التنفيذ ولكن قد يحذف Block ما زال يُستخدم بكثرة.
- أقل دقة من LRU.

3. Random:

- يتم اختيار Block عشوائياً للحذف بسيط جداً في العتاد ولكن غير دقيق و الأداء غير مضمون لكنه أحياناً يكون مقبولاً.

ملخص لما سبق:

- LRU: أدق وأفضل أداء
- FIFO: أسهل من LRU
- Random: أبسط وأقل تكلفة



Handling Writes (Write Policies)

❧ سياسات الكتابة تحدد كيف ومتى تُكتب البيانات إلى الذاكرة الرئيسية عند تنفيذ عمليات كتابة.

1. Write Through:

- نكتب في الكاش وفي الذاكرة الرئيسية فوراً.
- وتكون الذاكرة الرئيسية دائماً محدثة وصحيحة.
- وتصميم بسيط بطيء فالمعالج (CPU) قد ينتظر انتهاء الكتابة للذاكرة.

2. Write Back:

- نكتب في الكاش فقط ولا نكتب للذاكرة الرئيسية إلا عند استبدال الـ Block
- نحتاج إلى Dirty Bit:
 - إذا كان Dirty = 1 فيجب كتابة الـ Block للذاكرة قبل حذفه
 - إذا كان Dirty = 0 فلا حاجة للكتابة والذاكرة قد تكون قديمة مؤقتاً
- أسرع من Write Through ويقلل عدد مرات الكتابة للذاكرة ولكن الذاكرة الرئيسية قد تكون قديمة مؤقتاً والتصميم أعقد قليلاً.

3. Write Buffer - (تحسين الأداء):

- هو مخزن مؤقت تُوضع فيه عمليات الكتابة ويسمح لـ CPU بعدم الانتظار حتى تنتهي الكتابة للذاكرة ويُستخدم مع:
 - Write Through لتحسين السرعة
 - Write Back لتحسين الأداء أكثر و CPU لا ينتظر الكتابة

❧ أفضل أداء:

Write Back + Write Buffer

يقلل عدد الكتابات للذاكرة ويسمح لـ CPU بالاستمرار في التنفيذ ويعطي أفضل أداء عملياً في معظم المعالجات الحديثة.

Designing Memory System

الكاش وجد ليحل مشكلة:

■ CPU سريع جداً و Memory بطيئة فالكاش يقلل عدد مرات الوصول للذاكرة

ولكن حتى مع وجود الكاش:

أحياناً يصبح Cache Miss وعند الـ Miss يجب ان نرجع للذاكرة الرئيسية ولكن Miss Penalty كبير جداً لأن:

■ الـ Buss أبطأ من المعالج وحجم الـ Block أكبر من عرض الباص

مثال: Bus = 1 Word و Block = 4 Words

دورة ساعة (Clock Cycles) ≈ 65 Miss Penalty

فحتاج عدة دورات لجلب Block واحد أي استهلاك وقت طويل، Bandwidth ضعيف جداً.

تذكرة:

■ Bandwidth عرض الحزمة:

كمية البيانات التي يمكن نقلها خلال وحدة زمنية معينة
عادة تُقاس بـ Bytes/second أو Words/clock cycle

■ Bandwidth يحدد سرعة تدفق البيانات بين:

المعالج ↔ الكاش

الكاش ↔ الذاكرة الرئيسية

■ حتى لو كان زمن الوصول (Latency) قليل إذا كان الـ Bandwidth ضعيف فالأداء يكون سيئ.

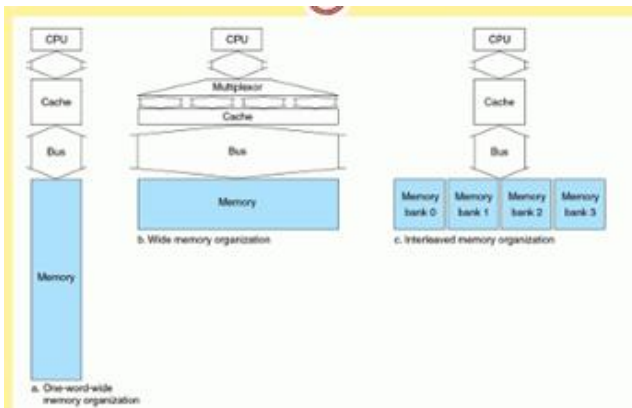
فكيف نقل وقت جلب الـ Block من الذاكرة؟

محاولة الحل الأولى: توسيع الباص (Widening the bus)

زيادة عرض الباص ليصبح 2 Words أو 4 Words بدل 1 Word

مما يقل Miss Penalty ويزيد Bandwidth

ولكن تكلفة أعلى وتعقيد أكبر في تصميم النظام



الحل الأفضل Interleaved Memory System

تقسيم الذاكرة الرئيسية إلى (Memory Banks) مستقلة وكل Memory Banks يستطيع العمل بالتوازي مع البنوك الأخرى.

كيف يعمل النظام؟

يتم إرسال عدة عناوين معاً إلى بنوك مختلفة وكل بنك يعالج طلبه في نفس الوقت ويتم استقبال البيانات Word بعد Word بشكل متتابع وسريع وهذا يعطي:

■ Bandwidth أعلى

■ Miss Penalty أقل

وبدون تغيير حجم الكاش أو الباص.

مثال:

Block = 4 Words ، الذاكرة مقسمة إلى 4 Memory Banks:

Bank0 → Word0

Bank1 → Word1

Bank2 → Word2

Bank3 → Word3

بينما Bank0 يرسل Word0، Bank1 يكون جاهزاً لإرسال Word1 والنتيجة تدفق مستمر للبيانات بدون انتظار طويل.

مثال:

حجم الـ Cache Block = 4 Words ، عرض بنك الذاكرة (DRAM Bank) = 1 Word الزمن المطلوب:

Bus cycle 1 لإرسال العنوان

Bus cycles 15 لكل وصول إلى DRAM

Bus cycle 1 لإرسال Word واحد من البيانات



الحالة الأولى: Bus بعرض 1 Word

حساب Miss Penalty

عند حدوث Cache Miss نحتاج:

1. إرسال العنوان مرة واحدة 1 cycle

2. جلب 4 Words من الذاكرة

كل Word يحتاج:

15 cycles للوصول إلى DRAM

1 cycle لإرسال البيانات

إذن لكل Word = 16 cycles

$$\begin{aligned} \text{Miss Penalty} &= 1 + 4 \times 15 + 4 \times 1 \\ &= 1 + 60 + 4 = 65 \text{ Bus Cycles} \end{aligned}$$

حساب Bandwidth

كل Word = 4 Bytes

Block = 4 Words → 16 Bytes

$$\text{Bandwidth} = 16 \text{ Bytes} / 65 \text{ Cycles} \approx 0.25 \text{ Bytes/Cycle}$$

→ ضعيف جداً Bandwidth

→ كبير Miss Penalty

الحل: Widening the Bus, Cache, and DRAM

أي زيادة عرض الباص والذاكرة بحيث يمكن نقل أكثر من Word في نفس الوقت.

الحالة الثانية: Memory Width = 2 Words

Miss Penalty

نحتاج عمليتي وصول بدل 4:

$$\begin{aligned} \text{Miss Penalty} &= 1 + 2 \times 15 + 2 \times 1 \\ &= 2 + 30 + 1 = 33 \text{ Cycles} \end{aligned}$$

Bandwidth

$$\text{Bandwidth} = 16 / 33 \approx 0.48 \text{ Bytes/Cycle}$$

→ أقل Miss Penalty

→ أعلى من الحالة الأولى Bandwidth

الحالة الثالثة: Memory Width = 4 Words

Miss Penalty

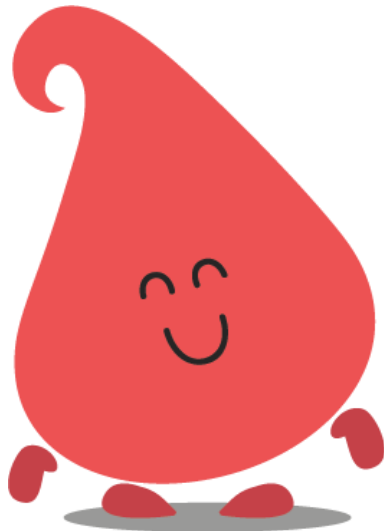
كل الـ Block يُجلب في وصول واحد:

$$Miss Penalty = 1 + 1 \times 15 + 1 \times 1 = 17 Cycles$$

Bandwidth

$$Bandwidth = 16 / 17 \approx 0.94 Bytes/Cycle$$

أداء يصبح ممتاز و Bandwidth قريب من المثالي



وبعد تجميع المعلومات وتنسيقها وتبسيطها لكم، إلى هنا نكون قد وصلنا إلى محطتنا الأخيرة في هذه المادة الشيقة فإن وجدتم فيها الخير فبفضل الله، وإن قصرنا معكم فالتمسوا لنا العذر

دتم مهندسين رائعين وقادة النور للأجيال اللاحقة ♥

فريق الـ RBCs ☺

The end ^-^